LAPORAN PRAKTIKUM

TEKNOLOGI IOT 2



Disusun Oleh :

Dhimaz dhewananda (25782037)

Dosen Pembimbing :

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T..

Agus Ambarwari, S.Pd., M.Kom

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2026

3.5.1 Praktikum 1: NodeMCU sebagai *Web Server* (Menyajikan HTML)

Skenario: Anda merakit NodeMCU sebagai server pintar. Modul ini membaca sensor DHT dan menampilkan suhunya di sebuah halaman web (UI), sekaligus menyediakan tombol di web tersebut untuk menghidupkan/mematikan alat via Modul Relay.

Langkah Kerja Rangkaian:

1. Hubungkan pin **DATA / OUT / S** DHT ke pin **D4 (GPIO 2)**. (Pastikan VCC DHT ke 3V3, GND ke GND).
2. Hubungkan pin **IN / Signal** Relay ke pin **D6 (GPIO 12)**. (Pastikan VCC Relay ke VIN/VU NodeMCU, GND ke GND).

Langkah Kerja Pemrograman:

1. Buat berkas baru (*New Sketch*). Tuliskan kode program di bawah. **Jangan lupa mengganti variabel ssid dan password!**
2. *Catatan teknis:* Kita menggunakan teknik **Raw String Literal** C++ (R"rawliteral(...)rawliteral") untuk menulis kode antarmuka HTML multibaris, sehingga kita tidak repot mengetikkan tanda kutip "" di setiap baris HTML.

```
#include <ESP8266WiFi.h>
#include <ESP8266WebServer.h>
#include <DHT.h>

const char* ssid = "Ramadhani";   const char*
password = "rama3311";

ESP8266WebServer server(80);

const byte dhtPin = 2;          const
byte relayPin = 12;
DHT dht(dhtPin, DHT11);

const char index_html[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>IoT Dashboard</title>
  <style>       body { font-family: Arial; text-align: center; margin-top: 50px; }       button { padding:
15px 30px; font-size: 20px; border-radius: 8px; margin: 10px; cursor: pointer;}
    .btn-on { background-color: #4CAF50; color: white; border: none; }
    .btn-off { background-color: #f44336; color: white; border: none; }
    .sensor-box { font-size: 24px; font-weight: bold; }
  </style>
</head>
<body>
```

```
  <h1>ESP8266 Web Server</h1>
  <div class="sensor-box">
    <p>Suhu Saat Ini: <strong>%TEMPERATURE%</strong> Celcius</p>
  </div>
  <h2>Kendali Relay</h2>
  <a href="/relay/on"><button class="btn-on">ON</button></a>
  <a href="/relay/off"><button class="btn-off">OFF</button></a>
</body>
</html>
)rawliteral";

void handleRoot() {
  String html = index_html; // Salin kerangka HTML ke variabel dinamis   float
t = dht.readTemperature();

  // Mengganti teks placeholder %TEMPERATURE% dengan suhu nyata   if
(isnan(t)) {
    html.replace("%TEMPERATURE%", "--");
  } else {
    html.replace("%TEMPERATURE%", String(t));
  }

  server.send(200, "text/html", html);
}

void handleRelayOn() {
digitalWrite(relayPin, HIGH);
server.sendHeader("Location", "/");      server.send(303);
}

void handleRelayOff() {
digitalWrite(relayPin, LOW);
server.sendHeader("Location", "/");      server.send(303);
}

void setup() {     Serial.begin(115200);      pinMode(relayPin,
OUTPUT);
digitalWrite(relayPin, LOW);     dht.begin();

  WiFi.mode(WIFI_STA);      WiFi.begin(ssid, password);      while
(WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
  Serial.println("\nIP Address Server Anda: ");
  Serial.println(WiFi.localIP());
```

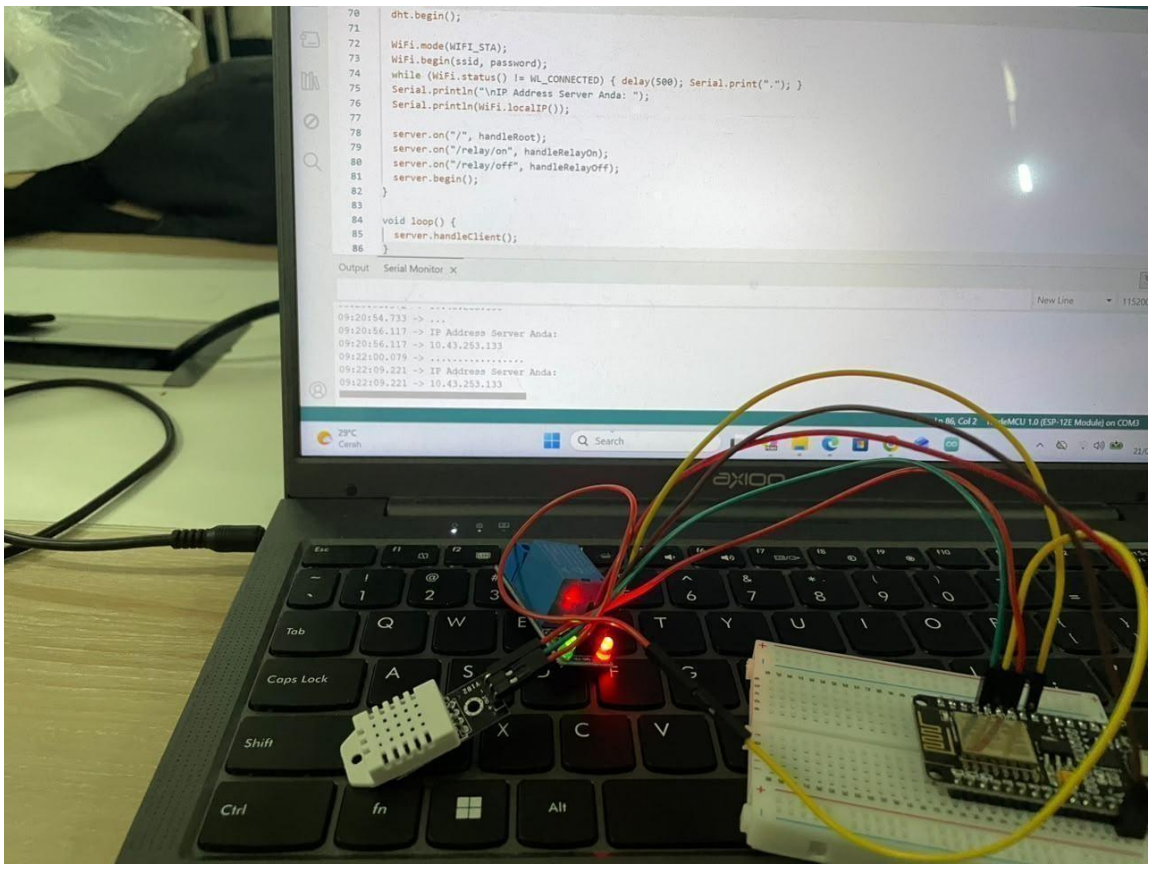
```
  server.on("/", handleRoot);       server.on("/relay/on",
handleRelayOn);      server.on("/relay/off",
handleRelayOff);     server.begin();     }

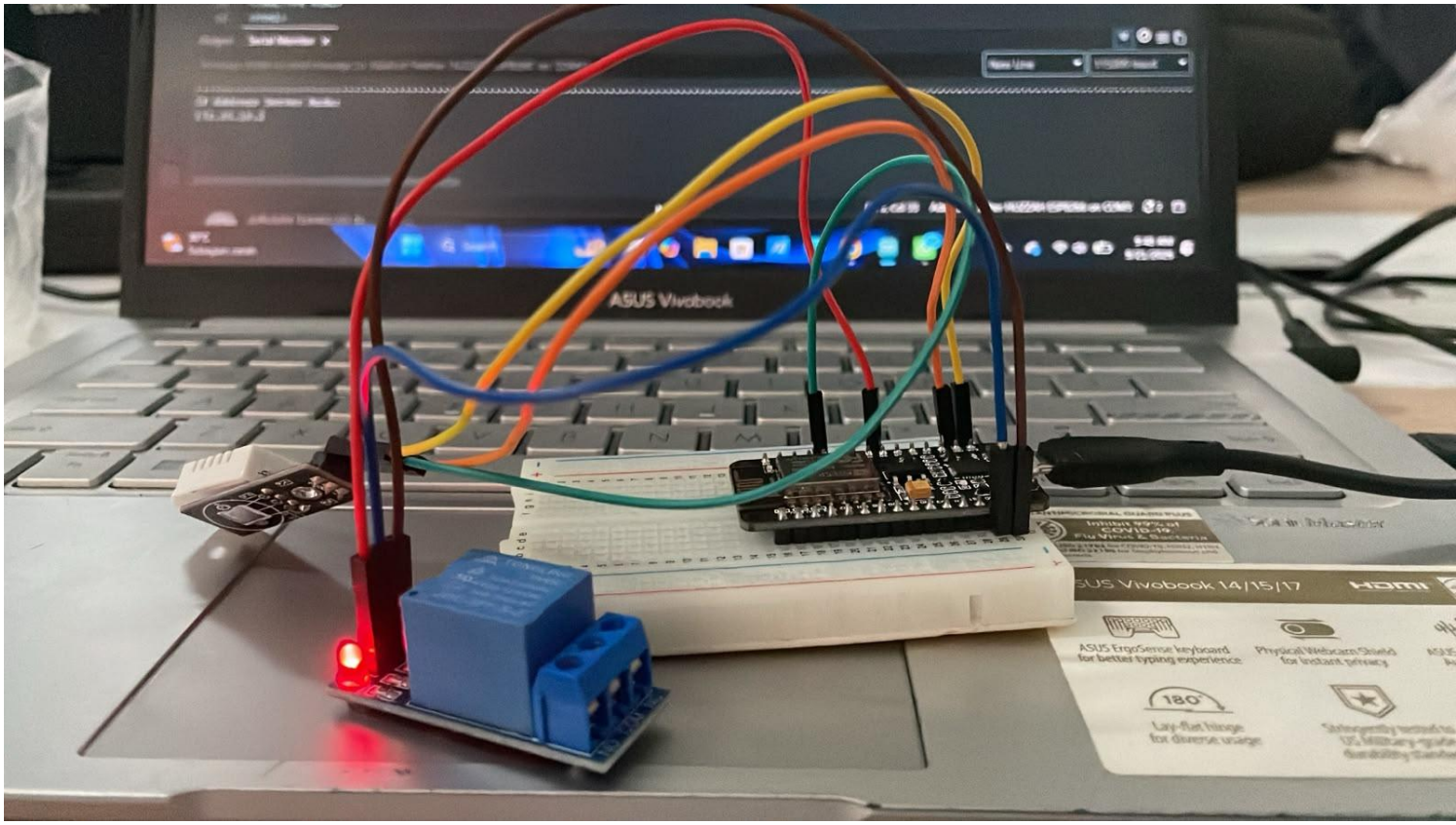
void loop() {
  server.handleClient();
}
```

3. **Upload** kode tersebut. Buka **Serial Monitor**, dan catat alamat IP yang didapat (contoh: 192.168.43.15).
4. Hubungkan Laptop/HP Anda ke Wi-Fi yang sama, buka *browser* (Chrome/Safari), lalu ketikkan alamat IP tersebut. Klik tombol ON dan OFF di layar untuk mendengarkan *Relay* berbunyi *cetek*.

Penjelasan Singkat Kode:

- ESP8266WebServer server(80);: Menginisialisasi *library Server* agar terus mendengarkan panggilan di jalur lalu lintas standar web (*Port 80*).
- html.replace("%TEMPERATURE%", String(t));: Fitur manipulasi teks (*String*). Perintah ini secara cerdas akan menyapu dokumen HTML, mencari teks *placeholder* %TEMPERATURE%, dan menggantinya dengan variabel angka suhu yang didapat dari sensor aktual sebelum dikirim ke layar pengunjung. Inilah esensi dari halaman *web* dinamis.
- server.on("/relay/on", handleRelayOn);: Fungsi *Routing*. Jika ada klien mengunjungi tautan URL berakhiran /relay/on, server akan menjalankan aksi mengubah *state* Relay, lalu me- *redirect* layar pengunjung kembali ke halaman depan (Location: /) via kode 303.
- server.handleClient();: Berada di dalam loop() untuk terus-menerus mengecek apakah ada permintaan pengunjung yang datang setiap milidetiknya.





3.5.2 Praktikum 2: NodeMCU sebagai *HTTP Client* (Meminta Layanan)

Skenario: Anda sekarang memprogram NodeMCU kedua (atau meminta teman kelompok di sebelah Anda) untuk menjadi *Client*. Perangkat Klien ini dipasangi LDR. Jika LDR tertutup kegelapan, ia akan mengirim sinyal HTTP secara jarak jauh (via jaringan Wi-Fi) untuk menyalakan Relay di *Server* milik teman Anda (Praktikum 1).

Langkah Kerja Rangkaian:

1. Rangkai sensor **LDR** dan **Resistor 10k Ohm** sebagai pembagi tegangan (seperti di Modul 2) lalu hubungkan jalur sinyalnya ke pin **A0**.

Langkah Kerja Pemrograman:

1. Buat berkas baru (*New Sketch*).
2. Tuliskan kode program di bawah. **Ganti variabel serverName dengan IP Address NodeMCU Praktikum 1 milik teman sebelah Anda.** (Pastikan Anda menggunakan Wi-Fi yang sama persis).

```
#include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>
#include <WiFiClient.h>

const char* ssid = "Ramadhani";    const
char* password = "rama3311";

const char* serverName = "http://172.20.10.4/relay/on";     const
byte ldrPin = A0;
```

```
void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
Serial.println("\nClient Terhubung ke Wi-Fi!");    }

void loop() {     int ldrValue =  analogRead(ldrPin);

  if ((WiFi.status() == WL_CONNECTED) && (ldrValue < 300)) {
    WiFiClient client;
    HTTPClient http;

    http.begin(client, serverName);        int
httpResponseCode = http.GET();

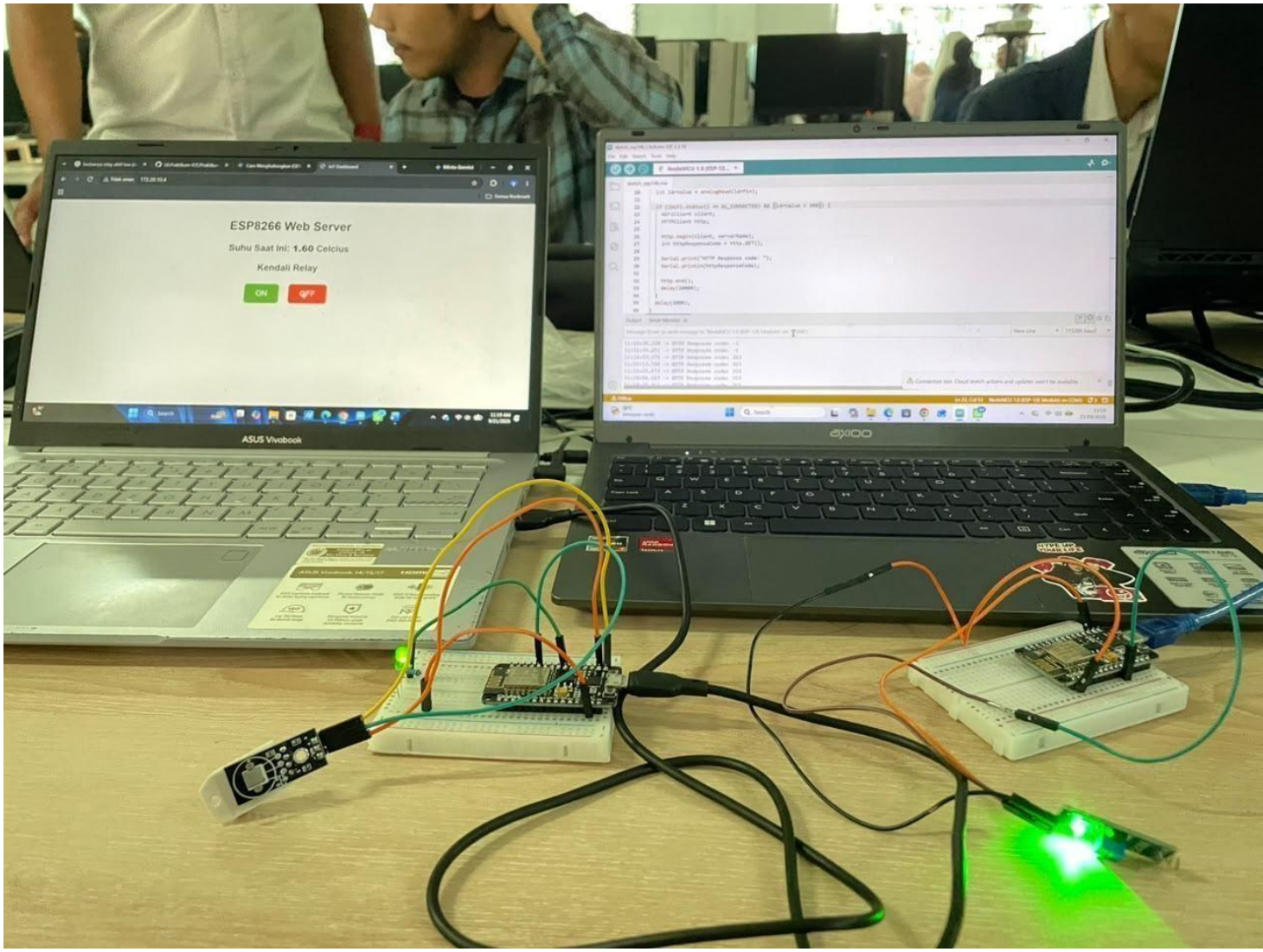
    Serial.print("HTTP Response code: ");
    Serial.println(httpResponseCode);

http.end();       delay(10000);
}
delay(2000);
}
```

3. **Upload** kode tersebut, dan buka Serial Monitor. Saat LDR Anda di *Client* ditutup tangan hingga gelap, perhatikan *Relay* di NodeMCU *Server* milik teman Anda pasti akan menyala!

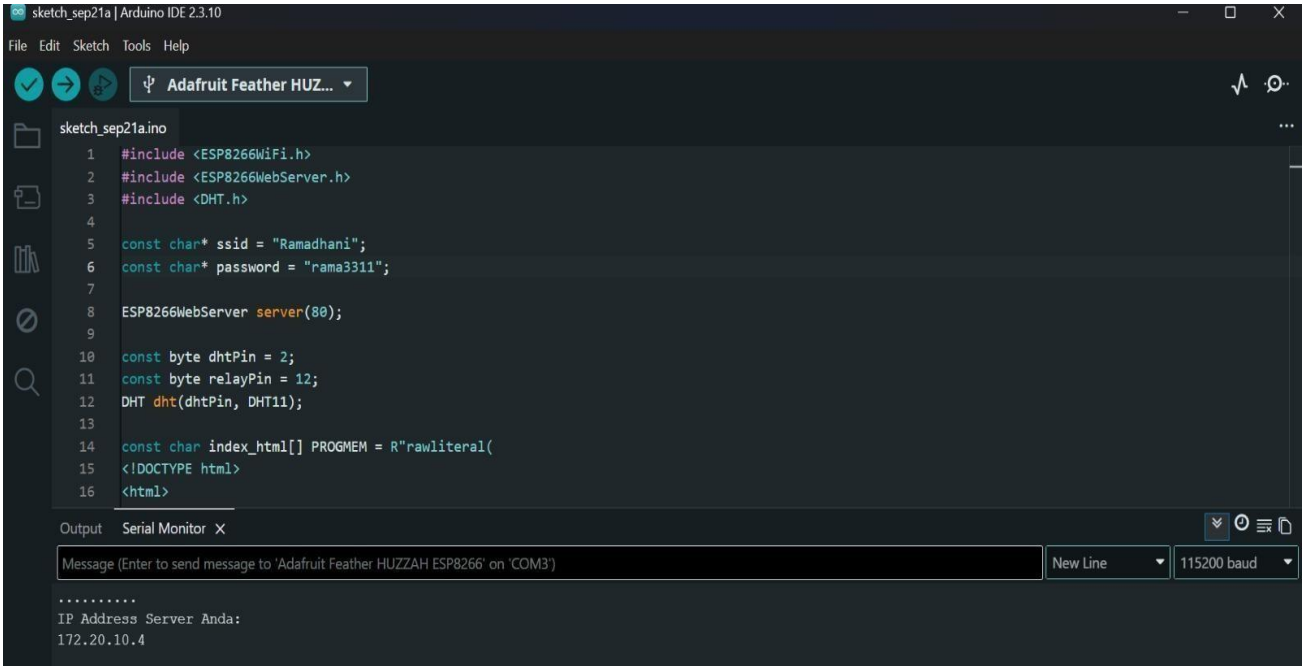
Penjelasan Singkat Kode:

- WiFiClient client; dan HTTPClient http;: Kelas pemanggil untuk memulai eksekusi permintaan *client* jarak jauh (mirip fungsi aplikasi *browser* di laptop).
- http.begin(client, serverName);: Menargetkan sasaran URL *server*.
- int httpResponseCode = http.GET();: Mengeksekusi ketukan pintu (*Request* HTTP GET). Jika Server merespons sukses, nilai variabel ini akan berisi angka standar jaringan seperti 200 (OK) atau 303 (Redirect). Jika URL rusak atau *Server offline*, variabel akan mereturn nilai negatif. • delay(10000);: *Cooldown* panjang 10 detik ini ditambahkan agar *Client* tidak memborbardir (*flood/DDoS*) Server dengan permintaan on secara terus-menerus setiap detik ketika kondisinya sedang gelap.

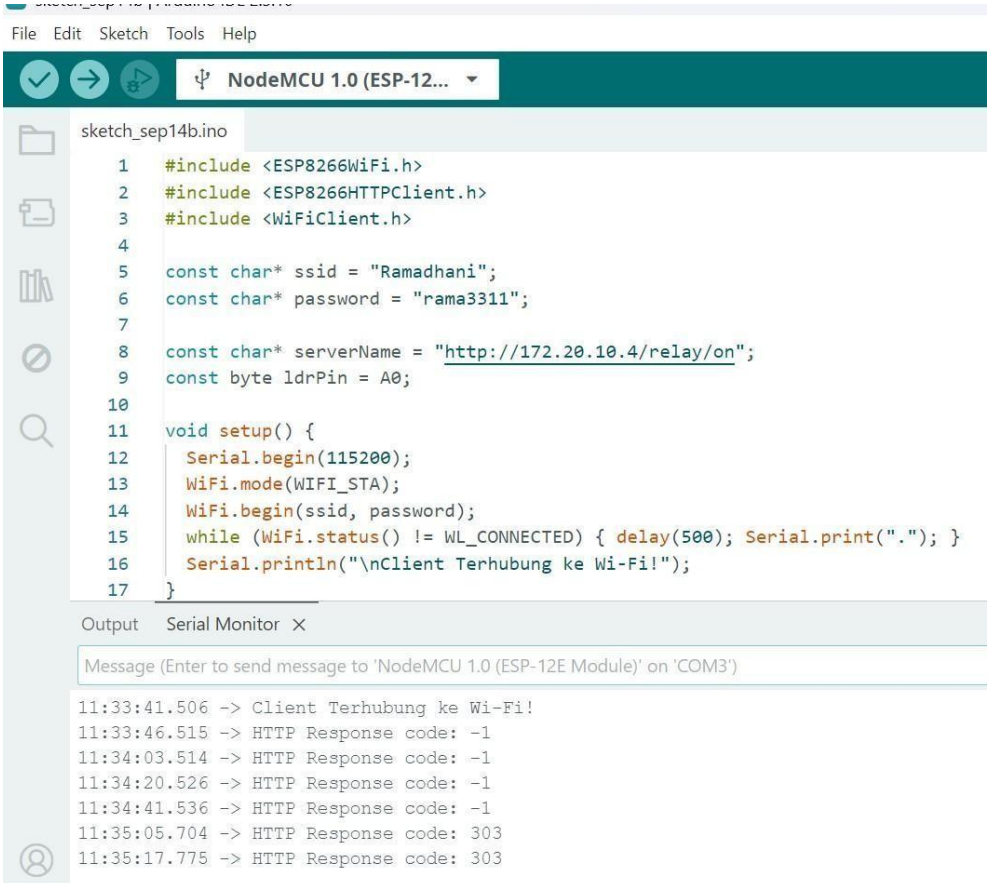


Ini Adalah dokumentasi dari server dan client cara kerja jika client memberikan Cahaya maka led pada server akan menyala otomatis dan jika mau mematikan led si server tinggal mencet of di dashboard pada server Scrnshoot pada client

Secrenshoot pada server



screnshoot pada client



Latihan lab03

Jawaban Latihan 1 (Pemahaman Jaringan)

Kita menggunakan mode jaringan **Station (STA)** pada kedua praktikum (baik NodeMCU Server maupun Client) karena kita menginginkan kedua perangkat tersebut terhubung ke satu jaringan perantara yang sama (seperti router Wi-Fi lokal atau hotspot HP).
Dalam mode STA, NodeMCU bertindak seperti "tamu" (seperti halnya laptop atau HP Anda) yang meminta alamat IP dari router Wi-Fi utama. Dengan cara ini, NodeMCU Server mendapatkan alamat IP lokal (misalnya 172.20.10.4), dan NodeMCU Client, yang juga berada di mode STA pada jaringan yang sama, dapat mengirim perintah HTTP langsung ke alamat IP tersebut.
Jika menggunakan mode Access Point (AP), salah satu NodeMCU harus memancarkan Wi-Fi sendiri (bertindak sebagai router). Ini akan membatasi fleksibilitas dan jangkauan, serta membuat perangkat sulit dipantau dari jaringan lokal utama kampus atau rumah.

Latihan 2 (Modifikasi Antarmuka & Telemetri): Pada Praktikum 1, tampilan web saat ini baru menyajikan data suhu ruangan dan masih menggunakan dua tombol terpisah (ON dan OFF). Lakukan dua modifikasi berikut pada *Web Server*:

i. **Kelembapan:** Tambahkan pembacaan kelembapan menggunakan fungsi dht.readHumidity(). Sisipkan placeholder %HUMIDITY% pada kerangka HTML di dalam tag (contoh: Kelembapan: %HUMIDITY% %), lalu perbarui fungsi handleRoot() menggunakan html.replace("%HUMIDITY%", ...) agar nilai kelembapan muncul di halaman *web*.

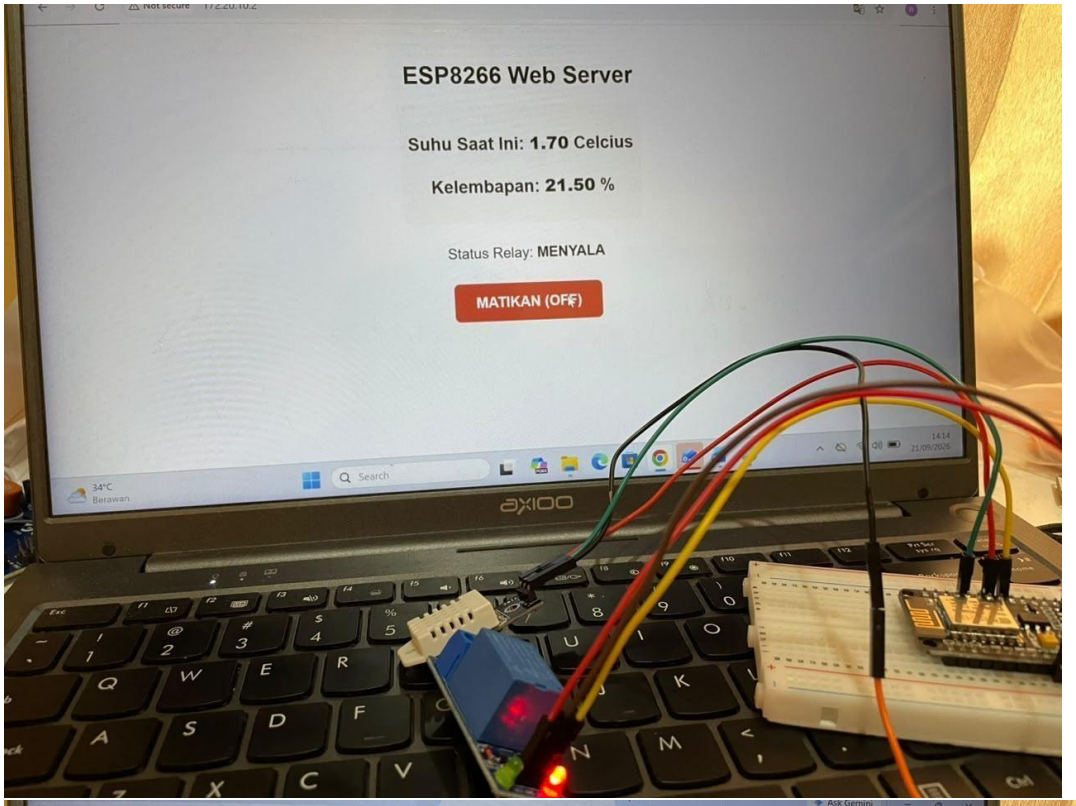
ii. **Tombol Dinamis Tunggal (*Toggle Button*):** Ubah antarmuka agar **hanya menggunakan 1 buah tombol tunggal** yang statusnya adaptif mengikuti kondisi **LED** saat ini:

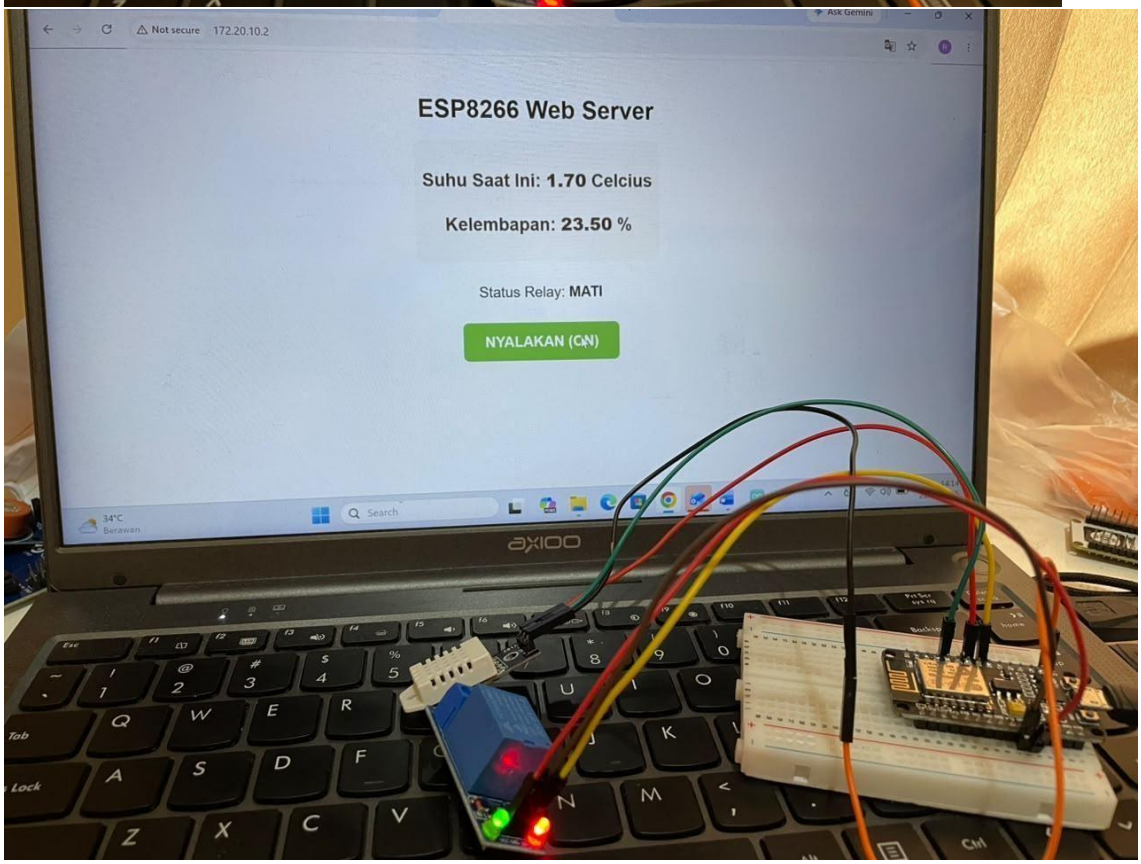
- Jika LED sedang **OFF**, tombol menampilkan tulisan **"NYALAKAN (ON)"** (misal: warna hijau/biru) dan mengarahkan aksi ke /relay/on.
- Jika LED sedang **ON**, tombol otomatis berubah menjadi tulisan **"MATIKAN (OFF)"** (warna merah) dan mengarahkan aksi ke /relay/off. *(Petunjuk: Anda dapat menyematkan placeholder baru di HTML seperti %RELAY_BUTTON%, lalu di fungsi handleRoot(), periksa status pin relay/LED menggunakan digitalRead(relayPin) sebelum mengganti placeholder tersebut dengan elemen tautan tombol yang sesuai).*

Tuliskan potongan baris kode HTML dan fungsi handleRoot() yang Anda ubah!

Untuk hasil praktikum saya lampirkan di bawah besertaa screenshot nya

Hasil dari Latihan 2


ESP8266 Web Server
Suhu Saat Ini: 1.70 Celcius
Kelembapan: 21.50 %
Status Relay: MENYALA
MATIKAN (OFF)


Not secure 172.20.10.2
ESP8266 Web Server
Suhu Saat Ini: 1.70 Celcius
Kelembapan: 23.50 %
Status Relay: MATI
NYALAKAN (ON)

Berdasarkan instruksi modifikasi untuk menambahkan indikator Kelembapan dan membuat Tombol Dinamis Tunggal (Toggle Button), berikut adalah potongan baris kode HTML dan fungsi handleRoot() yang telah diubah:

1. Potongan Modifikasi Kode HTML *(Menyisipkan placeholder %HUMIDITY% di dalam sensor-box dan mengganti dua tombol menjadi satu placeholder dinamis %RELAY_BUTTON%)*

HTML

```
<div class="sensor-box">
  <p>Suhu Saat Ini: <strong>%TEMPERATURE%</strong> Celcius</p>
  <p>Kelembapan: <strong>%HUMIDITY%</strong> %</p>
</div>

<h2>Kendali Relay</h2>
<!-- Placeholder ini akan diganti oleh sistem menjadi tombol ON atau OFF -->
%RELAY_BUTTON%
```

2. Potongan Modifikasi Fungsi handleRoot() *(Menambahkan pembacaan kelembapan dan logika if-else untuk mengecek status pin menggunakan digitalRead guna menampilkan tombol yang adaptif)*

C++

```
void handleRoot() {
  String html = index_html;
  float t = dht.readTemperature();
  float h = dht.readHumidity(); // Menambahkan fungsi pembacaan kelembapan

  // Mengganti teks placeholder Suhu dan Kelembapan dengan nilai nyata
  if (isnan(t) || isnan(h)) {
    html.replace("%TEMPERATURE%", "--");
    html.replace("%HUMIDITY%", "--");
  } else {
    html.replace("%TEMPERATURE%", String(t));
    html.replace("%HUMIDITY%", String(h));
  }

  // --- LOGIKA TOMBOL DINAMIS TUNGGAL (TOGGLE BUTTON) ---
  // Menggunakan digitalRead() untuk memeriksa kondisi Relay saat ini.
  // (Menggunakan logika Active-Low: jika pin bernilai LOW maka Relay menyala)
  if (digitalRead(relayPin) == LOW) {
    // Jika Relay sedang ON, tombol berubah menjadi warna MERAH (btn-off)
    // dan tautan diarahkan ke fungsi untuk MEMATIKAN relay (/relay/off)
    html.replace("%RELAY_BUTTON%", "<a href=\"/relay/off\"><button class=\"btn-off\">MATIKAN (OFF)</button></a>");
  } else {
    // Jika Relay sedang OFF, tombol berubah menjadi warna HIJAU (btn-on)
    // dan tautan diarahkan ke fungsi untuk MENYALAKAN relay (/relay/on)
    html.replace("%RELAY_BUTTON%", "<a href=\"/relay/on\"><button class=\"btn-on\">NYALAKAN (ON)</button></a>");
  }
```

Jawaban Latihan 3: Analisis Kode Client

Jika NodeMCU Server teman (target) mendadak kehilangan daya atau *offline*, maka Serial Monitor pada NodeMCU Client akan mencetak: **HTTP Response code: -1** *(atau nilai negatif lainnya, tergantung versi library).*

Analisis / Alasan Logis:

1. **Kegagalan Koneksi (Connection Refused/Timeout):** Saat Server mati, alamat IP target (contoh: 172.20.10.4) menjadi tidak dapat dijangkau di dalam jaringan Wi-Fi.
2. **Respon Internal Library:** Fungsi http.GET() dari library HTTPClient gagal "mengetuk pintu" Server. Karena tidak ada respon HTTP sama sekali dari server, library ESP8266 akan menghasilkan *error code* internal berupa angka negatif (biasanya -1), bukan kode status standar HTTP internet (seperti 200 untuk OK atau 303 untuk Redirect).

```
17:34:05.650 -> HTTP Response code: -11
17:35:40.548 -> .......
```

3. **Kesimpulan:** Angka minus menunjukkan bahwa masalahnya ada pada pengiriman jaringan (Server mati, IP target salah, atau tidak satu jaringan), bukan pada kode logika Client

Jawaban Latihan 4: Pengembangan Algoritma

Untuk mendesain Klien Pendeteksi Suhu, fungsi pembacaan LDR (analogRead) dihapus dan diganti dengan pembacaan sensor DHT (dht.readTemperature()). Modifikasi pada blok kondisional if ditambahkan syarat suhu > 35.0 dan validasi sensor !isnan(suhu). Berikut adalah baris algoritmanya: C++ void loop() {
// 1. Membaca data suhu dari sensor DHT float
suhu = dht.readTemperature();

// 2. Modifikasi conditional block (Suhu > 35 derajat)
// Pastikan WiFi terhubung, sensor terbaca dengan baik (!isnan), dan suhu di atas 35 Celcius if ((WiFi.status() == WL_CONNECTED) && (!isnan(suhu)) && (suhu > 35.0)) { WiFiClient client;
HTTPClient http;

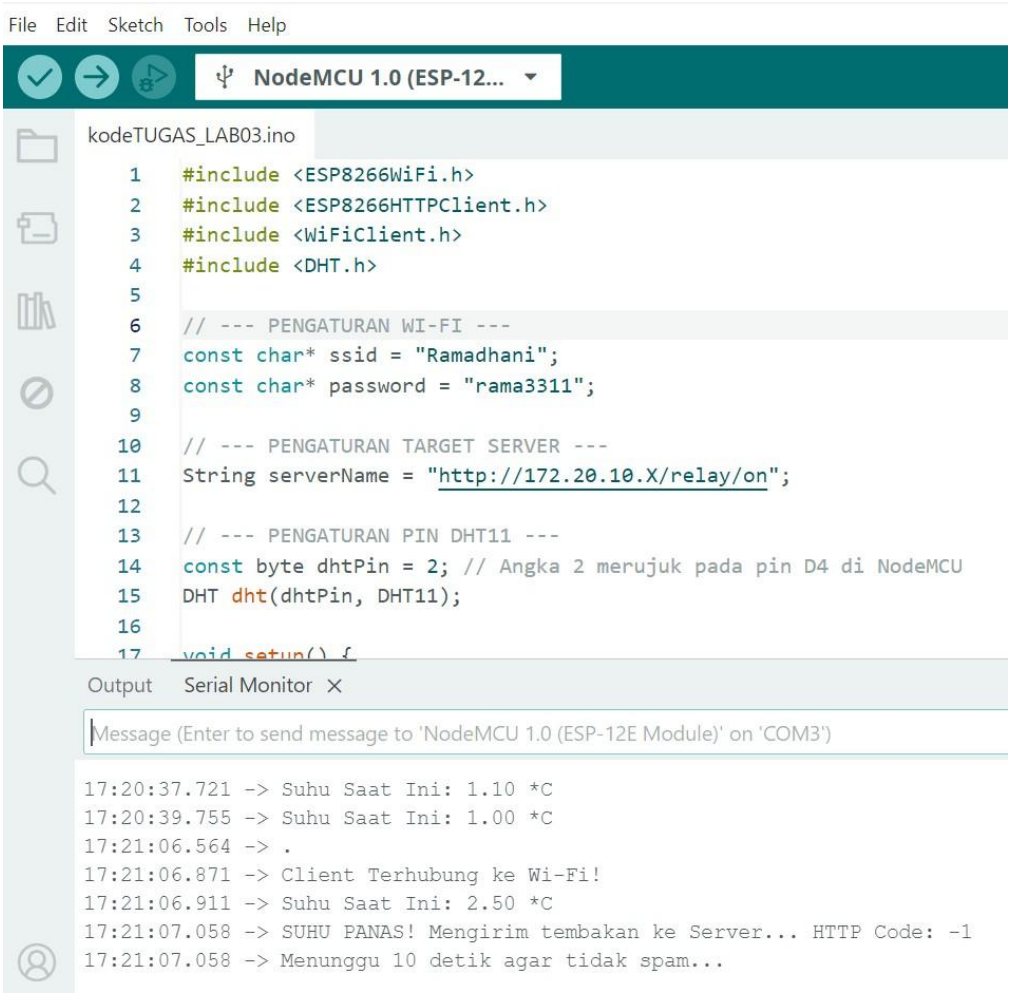
http.begin(client, serverName); int httpResponseCode = http.GET(); // Mengirim GET request ke Server

Serial.print("Suhu melebihi 2C! Mengirim sinyal peringatan. HTTP Response code: "); Serial.println(httpResponseCode);

http.end();

// Cooldown 10 detik agar tidak memborbardir (spam) Server delay(10000);
}

// Jeda 2 detik yang diwajibkan untuk sensor DHT sebelum membaca ulang data delay(2000);
}



Bisa di lihat disni outputnya akan memberikan peringatan lalu dia akan mengurim sinyal keserver Dan saya juga memberikan delay selama 10 detik agar tiddak terjadi spam.

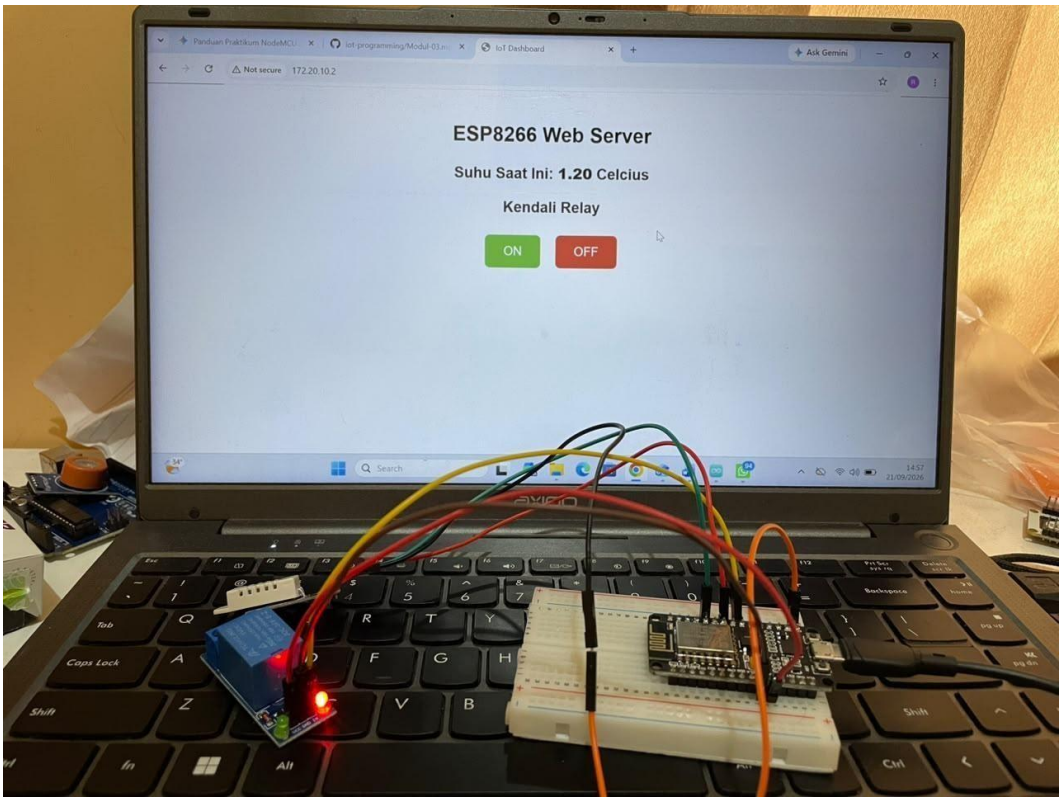
3.7 Tugas: Pengembangan *Web Server* Cerdas dan Analisis Arsitektur

Antarmuka (*User Interface*) IoT tidak boleh statis. Saat ini, *Web Server* kita tidak mampu memperbarui informasi suhunya secara otomatis ke layar pengunjung tanpa diperintah. Kita harus menyelesaikannya.

Skenario Proyek (*Web Dashboard*): Anda akan bertindak sebagai *Full-Stack IoT Developer* untuk memutakhirkan kapabilitas *Web Server* pada **Latihan 2**, sembari mengevaluasi limitasi arsitektur tradisional HTTP.

- **Tujuan:** Mampu memanipulasi struktur kerangka HTML mentah yang tersimpan di memori NodeMCU untuk mencapai interaktivitas tampilan otomatis (*Auto-Refresh*). • **Tingkat Kesulitan:** Menengah
- **Instruksi:**
  i. Lakukan modifikasi kode hanya pada sisi **NodeMCU Server (Latihan 2)**.
  ii. Modifikasi variabel Raw Literal index_html. Sisipkan perintah *Meta Refresh HTML* (sebuah Tag standar web kuno namun ampuh) persis di dalam cakupan *tag* <head> untuk memaksa *browser* memuat ulang halaman secara sekuensial setiap 5 detik.
  iii. *Hint (Petunjuk):* Sintaks standar yang harus disisipkan adalah <meta http-equiv="refresh" content="5">. iv. Setelah selesai, buka *browser*, dan amati layarnya. Biarkan selama 20 detik tanpa menyentuh *mouse/keyboard*.
- **Instruksi Analisis Arsitektur:**
  i. Setelah Auto-Refresh berjalan, cobalah tekan tombol *ON / OFF* (kendali LED) dari *browser*. ii. Apa kelemahan visual/interaksi (*flickering*) yang Anda alami saat menggunakan metode *Meta Refresh* HTTP konvensional seperti ini untuk sistem yang mengklaim diri sebagai *RealTime Dashboard*? Tulis evaluasi logis Anda di Laporan Praktikum (Anda bisa membahas masalah *bandwidth*, layar berkedip putih, atau potensi bentrokan jika Anda mengeklik tepat di detik ke-5).
- **Kriteria Keberhasilan:** ○ Nilai suhu dan kelembapan di *browser* otomatis berganti seiring kondisi lingkungan tanpa perlu mengeklik tombol *refresh* di *browser*.
  ○ Jawaban analisis mengenai kelemahan protokol HTTP GET statis tertuang jelas di dokumentasi tugas.

HASIL DARI TUGAS



1. Implementasi Modifikasi Untuk mencapai interaktivitas tampilan otomatis, kerangka HTML pada NodeMCU Server telah dimodifikasi dengan menyisipkan *Tag Meta Refresh* pada bagian <head>. Sintaks yang ditambahkan adalah <meta http-equiv="refresh" content="5">, yang memaksa *browser* untuk melakukan *HTTP GET Request* secara **otomatis setiap 5 detik.** Hasilnya, nilai pemantauan suhu dapat diperbarui tanpa intervensi manual dari pengguna.

2. Analisis Arsitektur dan Kelemahan HTTP Tradisional Meskipun modifikasi di atas berhasil menciptakan ilusi dasbor *real-time*, pendekatan menggunakan *Meta Refresh* HTTP konvensional ini memiliki batasan arsitektur yang fatal jika diterapkan pada sistem IoT industri. Berdasarkan pengujian yang dilakukan, terdapat tiga kelemahan utama:

- **Kelemahan Visual (Screen Flickering):** Setiap siklus 5 detik, layar *browser* akan berkedip putih sesaat (*flickering*). Hal ini terjadi karena *browser* harus menghancurkan (*destroy*) seluruh tampilan (DOM HTML) dan menggambarnya ulang dari nol. Tampilan visual yang tidak mulus ini menghasilkan *User Experience* (UX) yang buruk.
- **Kendala Interaksi (Collision / Bentrokan):** Terdapat potensi "tabrakan" (*collision*) antara aksi pengguna dan jadwal muat ulang halaman. Jika administrator menekan tombol kendali Relay (ON/OFF) tepat pada detik ke-5 saat halaman sedang melakukan *refresh*, maka perintah klik tersebut berisiko terabaikan (*dropped*) atau halaman menjadi tidak responsif.
- **Inefisiensi Bandwidth dan Beban Prosesor (Overhead):** Protokol HTTP bersifat *stateless*. Saat halaman dimuat ulang, NodeMCU harus merakit dan mengirimkan ulang keseluruhan struktur HTML, CSS, dan teks ke *browser*. Padahal, data aktual yang berubah hanyalah beberapa *byte* (nilai suhu). Pengiriman data statis secara berulang-ulang ini merupakan pemborosan *bandwidth* jaringan dan membebani kinerja memori/prosesor ESP8266 secara tidak efisien.

3. Kesimpulan Penggunaan protokol HTTP statis dengan metode *Meta Refresh* sangat tidak direkomendasikan untuk *Real-Time IoT Dashboard* berskala besar. Sebagai solusi di masa depan, arsitektur antarmuka harus ditingkatkan menggunakan teknologi **AJAX (Asynchronous JavaScript and XML)** atau **WebSockets**. Dengan teknologi tersebut, sistem dapat mengambil data suhu di latar belakang dan mengubah angka di layar secara spesifik tanpa perlu memuat ulang keseluruhan halaman web.